

Name of the Student	K. Lokesh Chandra
Reg. No	192425358
GitHub Link	https://github.com/Lokesh-123-06/Deep-Learning

SIMATS ENGINEERING

CO3 - ASSESSMENT TOOL 2

Game Based Learning

COURSE NAME: DEEP LEARNING

COURSE CODE: MLA04

SLOT :

COURSE OUTCOME COVERED

CO3: Analyze and apply convolutional neural network architectures, including convolutional layers, filters, parameter sharing, regularization, AlexNet and ResNet, to develop solutions for image classification and real-world computer vision applications. (L4)

CO Weightage: 25%

Duration: 1 Hour

Assessment Tool Weightage: 35%

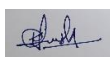
Marks: 50

Code of Conduct:

I K. Lokesh Chandra (192425358) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Name of the student: K. Lokesh Chandra

Criteria	Understanding of Concept (2.5)	Conceptual Clarity (2.5)	Problem-Solving Ability (2)	Solution Design & Application (2)	Teamwork & Game Participation (1)	Total (10)
Weightage	25 %	25%	20%	20 %	10 %	100%
Round 1						
Round 2						
Total						

Signature of the Faculty:

Total Marks :

QUESTIONS:01

Total Marks: 50

Question:

Design and participate in a two-round game-based learning activity to explore the fundamental concepts of Convolutional Neural Networks (CNNs), including architecture, layers, filters, parameter sharing, regularization, AlexNet, ResNet, and real-world applications. Progress through five levels in each round, applying your knowledge to solve challenges involving CNN design, feature extraction, architecture comparison, and practical applications.

The Game Progression

Round 1:

Basic CNN Concepts → Filters → Parameter Sharing → Architecture → AlexNet vs ResNet

Round 2:

Convolution → Feature Maps → Regularization → Architecture Comparison → Real-World Application

ROUND 1: CNN Architecture Builder (Learn by constructing and understanding CNN architectures layer by layer)

Level	Challenge	Description
Level 1	CNN Layer Matcher	Match CNN layers such as Convolution, Pooling, Flatten, and Fully Connected layers with their respective functions and roles in the network.
Level 2	Filter Finder	Identify suitable filters and kernels for feature extraction and understand how filter size, stride, and padding affect the output feature map.
Level 3	Parameter Sharing Puzzle	Determine how the same filter weights are reused across different image locations and calculate the number of parameters in a convolutional layer.
Level 4	CNN Architecture Debugger	Identify and correct problems in a CNN architecture, such as incorrect layer order, incompatible dimensions, and inappropriate layer selection.
Level 5	AlexNet vs ResNet Challenge	Compare AlexNet and ResNet architectures and identify how their architectural approaches differ, particularly the use of residual connections in ResNet.

ROUND 2: CNN Feature Detective (Learn by exploring how CNNs extract features, avoid overfitting, and solve real-world problems)

Level	Challenge	Description
Level 1	Edge Detection Explorer	Apply simple (3*3) filters to an image and determine how convolution operations can detect edges and other basic features.
Level 2	Feature Map Detective	Analyze feature maps produced by different CNN layers and identify the progression from simple features such as edges to complex patterns and objects.

Level	Challenge	Description
Level 3	Regularization Rescuer	Identify overfitting in a CNN model and select suitable regularization techniques to improve generalization and model performance.
Level 4	CNN Architecture Race	Compare ResNet and AlexNet for different image-classification applications and determine which architecture is more suitable for a given problem.
Level 5	CNN Application Master	Design a CNN-based solution for a real-world application such as medical image analysis, face recognition, object detection, or autonomous driving, and justify the selected architecture and layers.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Conceptual Clarity	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Problem-Solving Ability	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design & Application	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Teamwork & Game Participation	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

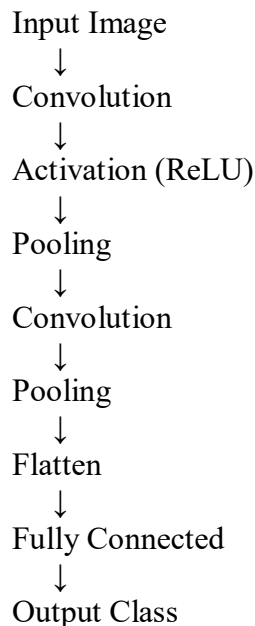
ROUND 1 – CNN Architecture Builder

Level 1 – CNN Layer Matcher

Answer

CNN Layer	Function
Convolution Layer	Applies filters to extract features such as edges, textures, and shapes
Pooling Layer	Reduces feature-map size and computational cost while retaining important features
Flatten Layer	Converts 2D/3D feature maps into a 1D vector
Fully Connected Layer	Uses extracted features to perform final classification

CNN Flow



Explanation

The **convolution layer** performs feature extraction. Early layers normally detect simple features such as edges, while deeper layers identify more complex patterns.

Pooling reduces spatial dimensions and helps reduce computation.

Flattening prepares the extracted features for the fully connected classification layer.

Level-4 Reasoning

The layers work together as a feature-learning pipeline. Removing convolution reduces feature extraction ability, while removing pooling can increase computational cost. Therefore, each layer has a specific role in the CNN architecture.

Level 2 – Filter Finder

The assessment asks to identify suitable filters and understand the effect of **filter size, stride, and padding** on the output feature map.

Answer

A filter/kernel is a small matrix that moves across an image and performs convolution.

Example edge-detection filter:

```
-1 -1 -1
 0  0  0
 1  1  1
```

This can detect horizontal edges.

Another filter:

```
-1  0  1
```

-1 0 1

-1 0 1

This can detect vertical edges.

Effect of Parameters

For an input of size:

32×32

with:

Filter = 3×3

Stride = 1

Padding = 0

Output size:

$$\begin{aligned}\text{Output} &= ((32 - 3 + 2(0)) / 1) + 1 \\ &= 30 \times 30\end{aligned}$$

If padding = 1:

$$\begin{aligned}\text{Output} &= ((32 - 3 + 2(1)) / 1) + 1 \\ &= 32 \times 32\end{aligned}$$

Key Points

- **Larger filter:** captures a larger local region.
- **Larger stride:** reduces output dimensions.
- **Padding:** helps preserve spatial dimensions.
- **3×3 filters:** commonly provide a good balance between feature extraction and computational cost.

Conclusion

Filter size, stride, and padding directly control the spatial dimensions and feature extraction behavior of a CNN.

Level 3 – Parameter Sharing Puzzle

The assessment requires understanding how the same filter weights are reused and calculating convolutional parameters.

Answer

Parameter sharing means that the same filter weights are applied at different positions of an image.

For example, a 3×3 filter:

w1 w2 w3

w4 w5 w6

w7 w8 w9

is moved across the image.

The same 9 weights are reused at every location.

Parameter Calculation

Suppose:

Input channels = 3

Number of filters = 16

Filter size = 3×3

Parameters per filter:

$$3 \times 3 \times 3 = 27$$

For 16 filters:

$$27 \times 16 = 432$$

If each filter has one bias:

$$\begin{aligned}\text{Total parameters} &= 432 + 16 \\ &= 448\end{aligned}$$

Without Parameter Sharing

If different weights were used for every image position, the number of parameters would become extremely large.

Importance

Parameter sharing:

- Reduces memory requirements.
- Reduces computational complexity.
- Allows the CNN to detect the same feature at different locations.
- Helps CNNs learn translation-related patterns.

Conclusion

Parameter sharing is one of the most important reasons CNNs are computationally practical for image processing.

Level 4 – CNN Architecture Debugger

The assessment asks to identify problems such as incorrect layer order, incompatible dimensions, and inappropriate layer selection.

Problem Architecture

Input Image



Flatten



Convolution



Pooling



Fully Connected

Problems

Problem 1: Flatten is placed before convolution.

A convolution layer expects spatial image information. Flattening first removes the 2D spatial structure.

Problem 2: Convolution should operate before flattening.

Problem 3: The architecture should progressively extract features before classification.

Correct Architecture

Input



Convolution



ReLU



Pooling



Convolution



ReLU



Pooling



Flatten



Fully Connected



Output

Reasoning

The corrected design first extracts spatial features, then reduces their dimensions, and finally converts the learned representation into a vector for classification.

Conclusion

The correct order is important because CNNs depend on spatial relationships during feature extraction.

Level 5 – AlexNet vs ResNet Challenge

The assessment specifically requires comparison of AlexNet and ResNet, especially the use of residual connections.

Answer

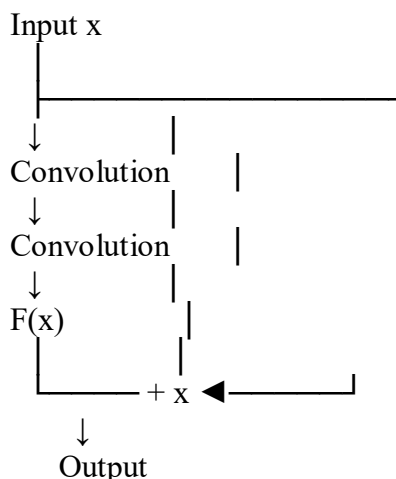
Feature	AlexNet	ResNet
Architecture	Traditional deep CNN	Residual CNN
Main innovation	Deep CNN with ReLU, pooling and large-scale image classification	Residual/skip connections
Depth	Relatively shallow compared with modern networks	Can be very deep
Skip connections	No	Yes
Training deep networks	More difficult	Easier
Vanishing-gradient problem	More significant	Reduced
Feature learning	Good	Very strong
Suitable for	Basic/older image-classification tasks	Large and complex vision tasks

AlexNet

AlexNet became an important CNN architecture for large-scale image classification. It uses convolutional layers followed by fully connected layers.

ResNet

ResNet introduces **residual connections**:



Instead of directly learning:

$H(x)$

the residual block learns:

$F(x) = H(x) - x$

and produces:

$H(x) = F(x) + x$

Conclusion

For modern deep image-classification problems, **ResNet is generally more suitable** because residual connections make it easier to train deeper networks.

ROUND 2 – CNN Feature Detective

The second round focuses on convolution, feature maps, regularization, architecture comparison, and real-world applications.

Level 1 – Edge Detection Explorer

Answer

Consider a simple image:

10 10 10

10 10 10

10 10 10

and an edge-detection filter:

-1 0 1

-1 0 1

-1 0 1

The filter is moved over different image locations.

Convolution Process

At each position:

Output = Sum(Input × Filter)

For a region with a strong change from dark to bright, the output becomes large.

Interpretation

- **Large positive/negative value:** strong edge.
- **Value near zero:** weak/no edge.
- Different filters detect different orientations and patterns.

Conclusion

CNN convolution converts raw pixel information into useful feature maps containing edges and other visual patterns.

Level 2 – Feature Map Detective

The assessment asks to identify the progression from simple features to complex patterns and objects.

Answer

CNN layers learn features progressively.

Input Image



Early Layers



Edges / Lines



Middle Layers



Textures / Shapes



Deep Layers



Parts of Objects



Final Layers



Complete Objects

Example – Face Recognition

Early layer:

Edges

Middle layer:

Eyes

Nose

Mouth

Deep layer:

Face structure

Analysis

Early layers preserve more spatial information and learn simple patterns.

As depth increases, the receptive field becomes larger and the network combines simple features into complex representations.

Conclusion

CNNs automatically learn a **hierarchical representation**, moving from low-level features to high-level semantic features.

Level 3 – Regularization Rescuer

The assessment requires identifying overfitting and selecting suitable regularization techniques.

Problem

Suppose:

Training Accuracy = 98%

Validation Accuracy = 72%

This indicates **overfitting**.

The model performs very well on training data but poorly on unseen data.

Suitable Solutions

1. Dropout

Randomly disables some neurons during training.

Input → CNN → Dropout → Fully Connected → Output

This prevents the network from depending too strongly on particular neurons.

2. Data Augmentation

Create variations of training images:

- Rotation
- Flipping
- Cropping
- Scaling
- Brightness changes

3. L2 Regularization

Adds a penalty for large weights:

$$\text{Loss} = \text{Original Loss} + \lambda \sum w^2$$

4. Early Stopping

Stop training when validation performance stops improving.

Recommendation

For a small image dataset:

Data Augmentation

+

Dropout

+

Early Stopping

is a practical combination.

Conclusion

Regularization improves generalization by preventing the CNN from memorizing the training data.

Level 4 – CNN Architecture Race

The assessment asks to compare ResNet and AlexNet for different image-classification applications and select the suitable architecture.

Scenario 1: Small Simple Dataset

Choice: AlexNet-style simpler CNN

Reason: A smaller model can reduce computational requirements and may be sufficient for a simple classification problem.

Scenario 2: Large Complex Dataset

Choice: ResNet

Reason:

- Deeper architecture.
- Residual connections.
- Better feature representation.
- Easier training of deep networks.

Scenario 3: Complex Object Recognition

Choice: ResNet

Complex images



Deep feature extraction



Residual blocks



Classification

Comparison

Requirement	Better Choice
Simple classification	AlexNet-style architecture
Large dataset	ResNet
Very deep network	ResNet
Complex visual patterns	ResNet
Modern computer vision	ResNet

Final Decision

For most modern large-scale image-classification applications, **ResNet is the stronger choice** because residual connections allow deeper and more effective feature learning.

Level 5 – CNN Application Master

The assessment allows applications such as **medical image analysis, face recognition, object detection, and autonomous driving.**

Proposed Application: Medical Image Classification

Problem

Design a CNN system to classify medical images into:

Normal

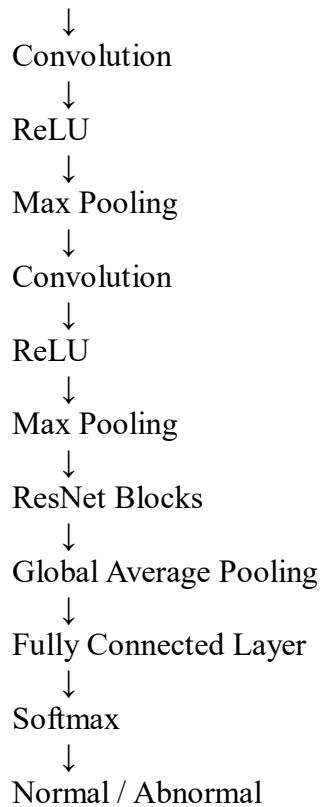
Abnormal

Proposed Architecture

Medical Image



Resize / Normalize

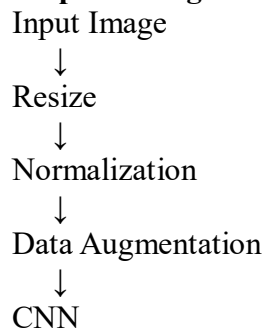


Why ResNet?

ResNet is suitable because:

- It can learn deep visual features.
- Residual connections help train deeper networks.
- It can identify complex patterns in medical images.
- Transfer learning can be used when the medical dataset is limited.

Preprocessing



Regularization

To reduce overfitting:

Data Augmentation

+

Dropout

+

Early Stopping

Evaluation

The model should be evaluated using:

Accuracy

Precision

Recall

F1-Score

Confusion Matrix

For medical classification, **recall/sensitivity is particularly important** because missing an abnormal case can be more serious than producing an additional false positive.

Practical Application

The trained model can assist healthcare professionals by automatically identifying suspicious images for further examination.

Final Solution

Medical Dataset



Preprocessing



ResNet-based CNN



Feature Extraction



Classification



Normal / Abnormal



Performance Evaluation

Conclusion: A ResNet-based CNN with appropriate preprocessing and regularization provides a practical solution for medical image classification while maintaining strong feature-learning capability.

Game Participation / Presentation

Since the assessment itself is a **game-based learning activity**, you can write the following for the participation component:

Round 1

Level 1 → Identified CNN layers

Level 2 → Selected filters and calculated feature-map dimensions

Level 3 → Calculated convolution parameters

Level 4 → Debugged CNN architecture

Level 5 → Compared AlexNet and ResNet

Round 2

Level 1 → Performed edge detection

Level 2 → Analyzed feature maps

Level 3 → Identified and reduced overfitting

Level 4 → Selected suitable CNN architecture

Level 5 → Designed a real-world CNN solution

Overall Learning Outcome

Through the two rounds, the activity demonstrates the ability to:

- Understand CNN architecture.
- Explain convolution and filters.
- Calculate parameters using parameter sharing.
- Analyze feature maps.
- Identify overfitting.
- Compare AlexNet and ResNet.
- Select suitable architectures for applications.
- Design a CNN solution for a real-world problem.

This directly addresses **CO3**, which focuses on analyzing and applying CNN architectures, convolutional layers, filters, parameter sharing, regularization, AlexNet, ResNet, and real-world computer-vision applications.